NIALL CASSIDY

DEVELOPMENTAL TESTING

A2 WJEC COMPUTER SCIENCE

NIALL CASSIDY CANDIDATE NO: 0109
CENTER NO: 71741

Section 7: Developmental Testing (5/100 marks

- provided evidence of comprehensive testing at each stage of the development of the solution to the chosen problem
- provided evidence of all problems encountered and fully justified and evidenced the actions taken to overcome these problems

7.1 Developmental Testing

When developing the project, in all stages, I encountered unintended outcomes and errors when running the code. I made it a priority to fix the debug these challenges as they arose to ensure a bug free program to Mulligan Motors.

Below I have listed an example of some of the issues that took considerate changes or problem-solving skills to fix the programming defects in my code. Further in the document I will go into each error in detail, specifying what caused the problem and the effectiveness of my solution to fix

Issues:

1) Overcrowding in the main menu.

2) Format of generated reports.

3) Referential Integrity errors.

4) Functionality of Combo Boxes

5) Email Validation error

7.1.1 Overcrowding in the Main Menu



After adding Mulligan Motor logo to the manager homepage, the size of the picture and amount of buttons on screen became overcrowded. Even when opened in full screen (as seen on the left) certain buttons, like the log off functionality, were not to be found.

This was a unique issue to the **manager** home screen. Accounts with 'sales' access simply didn't have enough buttons and access to tables for this problem to occur.

```
Button(manage_frame, text="Manage Orders", font=("Arial", 12),
    command=lambda: manage_table(
        "Orders",
        ["OrderID", "OrderValue", "DatePlaced", "DateArriving", "CustomerID", "VehicleID"],
        """
        INSERT INTO Orders (OrderValue, DatePlaced, DateArriving, CustomerID, VehicleID)
        VALUES (?, ?, ?, ?, ?)
        """,
        """
        UPDATE Orders
        SET OrderValue = ?, DatePlaced = ?, DateArriving = ?, CustomerID = ?, VehicleID = ?
        WHERE OrderID = ?
        """,
        m_window
    )
).pack(pady=5)
```

```
127    # A separate frame for utility buttons
128    utility_frame = Frame(m_window, bg="#87CEEB")
129    utility_frame.pack(pady=10)
130
131    Button(utility_frame, text="Generate Report", font=("Arial", 12),
132           command=lambda: generate_report(m_window),
133           bg="#2ecc71", fg="white").pack(pady=5)
134
135    Button(utility_frame, text="Backup Database", font=("Arial", 12),
136           command=backup_database, bg="#4682B4", fg="white").pack(pady=5)
137
138    Button(utility_frame, text="Help", font=("Arial", 12),
139           command=help_page, bg="#4682B4", fg="white").pack(pady=5)
140
141    # Log Off button at the bottom
142    Button(m_window, text="Log Off", font=("Arial", 12),
143           command=m_window.destroy, bg="#f39c12", fg="white"
144    ).pack(side="bottom", pady=20)
145
146    m_window.mainloop()
```

This highlights my overly simple code that simply involved packing all buttons vertically on top of one another. This is not very scalable or aesthetically pleasing. If I wanted to add anymore processes to the menus, it wouldn't be possible in its current state.

Solution



My solution involved revamping the framework for the main menu and changing it into a dual-column design. Whilst this strays against the original form design, I believes it offers a more practical, scalable, and aesthetically pleasing alternative to the single stack format used before.

Below is the new code pieces for each row in the menu. Compared to the old code, it is more complex and lengthy.



ROW 0

VEHICLE STOCK / CUSTOMERS



ROW 1

ORDERS / STAFF



ROW 2

TEST DRIVES / ACCESS

```
    # Row 3
    btn_payslips = Button(grid_frame, text="Manage Payslips", font=("Arial", 12),
        command=lambda: manage_table(
            "Payslips",
            ["PayslipID", "StaffID", "HoursWorked", "HourlyRate", "TotalOwed"],
            '''
            INSERT INTO Payslips (StaffID, HoursWorked, HourlyRate, TotalOwed)
            VALUES (?, ?, ?, ?)
            ''',
            '''
            UPDATE Payslips
            SET StaffID = ?, HoursWorked = ?, HourlyRate = ?, TotalOwed = ?
            WHERE PayslipID = ?
            ''',
            m_window
```

ROW 3

PAYSLIPS

```
    # Utility Buttons
    # We'll put them on row 3 or 4, column 1 or 0
    btn_report = Button(grid_frame, text="Generate Report", font=("Arial", 12),
        command=lambda: generate_report(m_window),
        bg="#2ecc71", fg="white"
    )
    btn_report.grid(row=3, column=1, padx=10, pady=5, sticky="ew")

    btn_backup = Button(grid_frame, text="Backup Database", font=("Arial", 12),
        command=backup_database, bg="#468284", fg="white"
    )
    btn_backup.grid(row=4, column=0, padx=10, pady=5, sticky="ew")

    btn_help = Button(grid_frame, text="Help", font=("Arial", 12),
        command=help_page, bg="#468284", fg="white"
    )
    btn_help.grid(row=4, column=1, padx=10, pady=5, sticky="ew")

    # Log Off button on row 5, col 1, to place it bottom-right
    btn_logoff = Button(grid_frame, text="Log Off", font=("Arial", 12),
        command=m_window.destroy, bg="#f39c12", fg="white"
    )
    btn_logoff.grid(row=5, column=1, padx=10, pady=10, sticky="e")

    # Make columns expand a bit if we resize the window
    grid_frame.columnconfigure(0, weight=1)
    grid_frame.columnconfigure(1, weight=1)

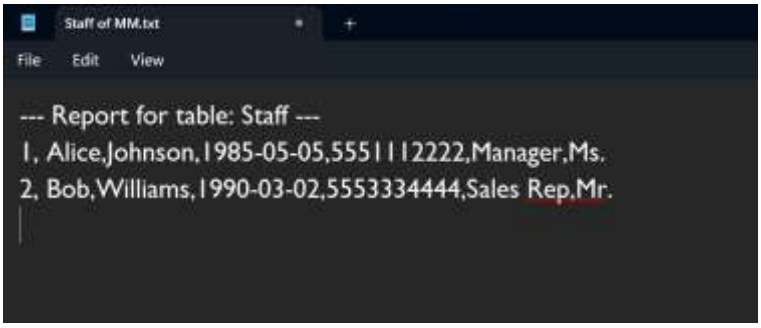
    m_window.mainloop()
```

UTILITY BUTTONS

GENERATE REPORT / BACKUP / LOGOFF/ HELP MENU

7.1.2 Report Generation Format

Part of the utility I wanted my program to provide was a report generation feature. This was exporting a table's contents to a text file that could, for example, shared externally to another computer. Whilst this was achieved, the initial format of the text file was unreadable and did not meet the standard I wanted to achieve.

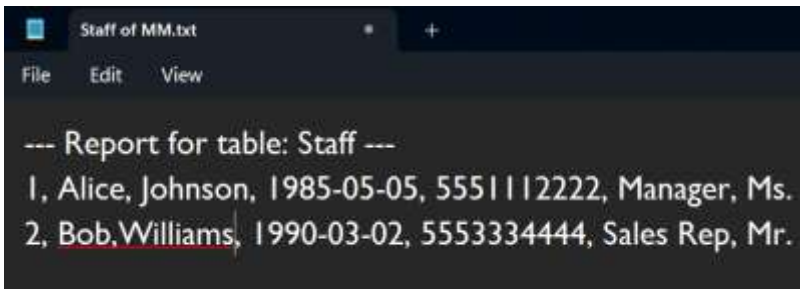


The output for the text file did not space the txt which was an unintended bug within the code. Although there were commas differentiating between attributes, fields such as DOB, Age, and Telephone became indistinguishable



The code responsible for the formatting issues can be seen to the left.

Solution



After fixing my code, I achieved a more professional and spaced format for my reports.



```
        try:
            conn = sqlite3.connect('mulligan_motors.db')
            cursor = conn.cursor()
            cursor.execute(f"SELECT * FROM {chosen_table}")
            rows = cursor.fetchall()
            conn.close()

            with open(file_path, 'w', encoding='utf-8') as f:
                f.write(f"--- Report for table: {chosen_table} ---\n")
                for row in rows:
                    row_str = ", ".join(str(item) for item in row)
                    f.write(row_str + "\n")

            messagebox.showinfo("Success", f"Report generated: {file_path}")
        except Exception as e:
            messagebox.showerror("Error", f"Could not generate report: {e}")
```

The reason why my code was unspaced was simple. There was a typo in a single line of code were I specified what should come after each attribute. It was missing a space so it was coded like （"，"） instead of （", "）This subtle difference was hard to find but has reminded me of the time cost of simple mistakes in my code. Since this incident, I tried to reduce the amount of simple typo mistakes in my code.

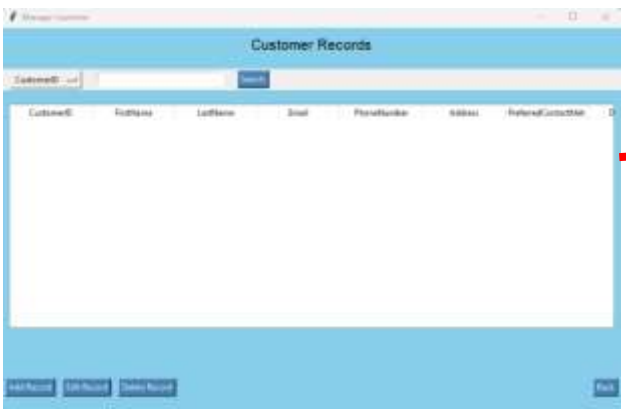
7.1.3 Referential Integrity

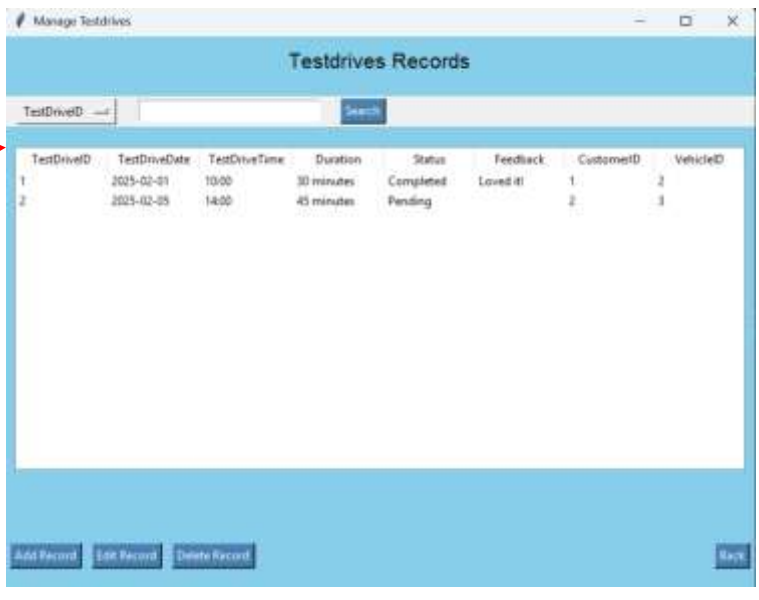
When testing the referential capability of the program, I discovered an issue which dampened the overall data integrity of the system.

If a customer wanted to delete their data on Mulligan Motor's database, for example, their linked test drives would remain. The result is that the test drives record would frequently refer to customers who didn't exist. This would cause confusion to users of the system especially those inexperienced with the software and had to be fixed.

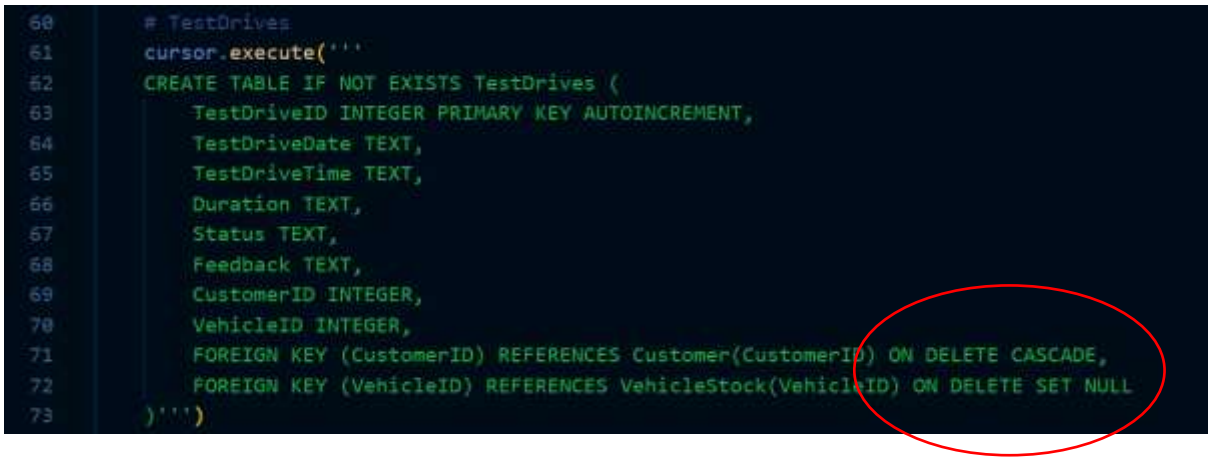
```
# TestDrives
cursor.execute('''
CREATE TABLE IF NOT EXISTS TestDrives (
    TestDriveID INTEGER PRIMARY KEY AUTOINCREMENT,
    TestDriveDate TEXT,
    TestDriveTime TEXT,
    Duration TEXT,
    Status TEXT,
    Feedback TEXT,
    CustomerID INTEGER,
    VehicleID INTEGER,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (VehicleID) REFERENCES VehicleStock(VehicleID)
```

Despite all customer records being deleted, no test drive records still referred to now deleted customers





Solution



```
60      # TestDrives
61      cursor.execute('''
62      CREATE TABLE IF NOT EXISTS TestDrives (
63          TestDriveID INTEGER PRIMARY KEY AUTOINCREMENT,
64          TestDriveDate TEXT,
65          TestDriveTime TEXT,
66          Duration TEXT,
67          Status TEXT,
68          Feedback TEXT,
69          CustomerID INTEGER,
70          VehicleID INTEGER,
71          FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID) ON DELETE CASCADE,
72          FOREIGN KEY (VehicleID) REFERENCES VehicleStock(VehicleID) ON DELETE SET NULL
73      )''')
```

After searching Stack Overflow to solutions to this issue, I eventually deduced that my DB Setup.py file was missing key elements in each of it's tables. At the end of each line, I was missing the phrases:

"ON DELETE CASCADE/NULL"

These lines inform the system what to do in case the foreign keys it refers to no longer exists. Now this is in place, the system reacted appropriately to missing references and referential integrity was restored.

7.1.4 Functionality of Combo Boxes



When implementing Combo Boxes into my solution, I noticed a bug I have never seen before. Users would see an unclickable dropdown containing something like ["Field1", "Field2", ...] instead of separate options. As a result, the drop down box became unusable.

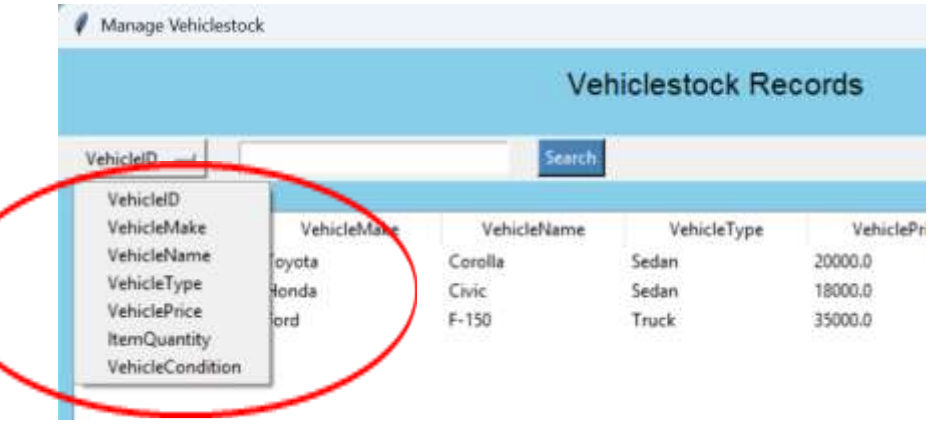
```
202     # Search area
203     search_frame = ttk.Frame(table_window)
204     search_frame.pack(pady=10, fill="x")
205
206     dropdown_var = StringVar()
207     dropdown_var.set(fields[0])
208     OptionMenu(search_frame, dropdown_var, fields).pack(side="left", padx=10)
209
210     search_entry = Entry(search_frame, font=("Arial", 12), width=20)
211     search_entry.pack(side="left", padx=10)
212     Button(search_frame, text="Search", command=search,
213            bg="#468284", fg="white").pack(side="left", padx=10)
214
```

The code responsible can be seen to the left. As I figured, I forgot * (unpacking operator) from *fields in the OptionMenu creation at line #208. This means instead of passing individual field options, it passes the entire list as a single menu item..

Solution

```
202     # Search area
203     search_frame = ttk.Frame(table_window)
204     search_frame.pack(pady=10, fill="x")
205
206     dropdown_var = StringVar()
207     dropdown_var.set(fields[0])
208     OptionMenu(search_frame, dropdown_var, *fields).pack(side="left", padx=10)
209
210     search_entry = Entry(search_frame, font=("Arial", 12), width=20)
211     search_entry.pack(side="left", padx=10)
212     Button(search_frame, text="Search", command=search,
213            bg="#468284", fg="white").pack(side="left", padx=10)
```

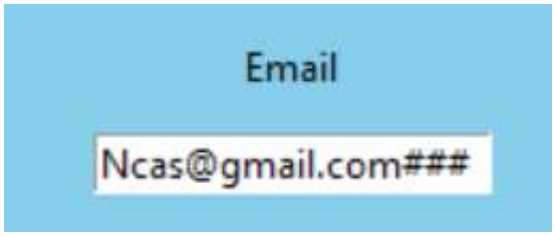
As you can see, I added back the * in line #208 after comparing my code with relentless examples of code for combo boxes and finally noticing the difference. Without an *, the system would display python list syntax without proper options.



Resultingly, the drop down box now works as expected. The user can click any of the options presented.

7.1.5 Email Validation error

An issue I had when developing the validation module of my code is that seeming valid emails with invalid endings to them (e.g ### as shown) would be entered and saved successfully in the database.

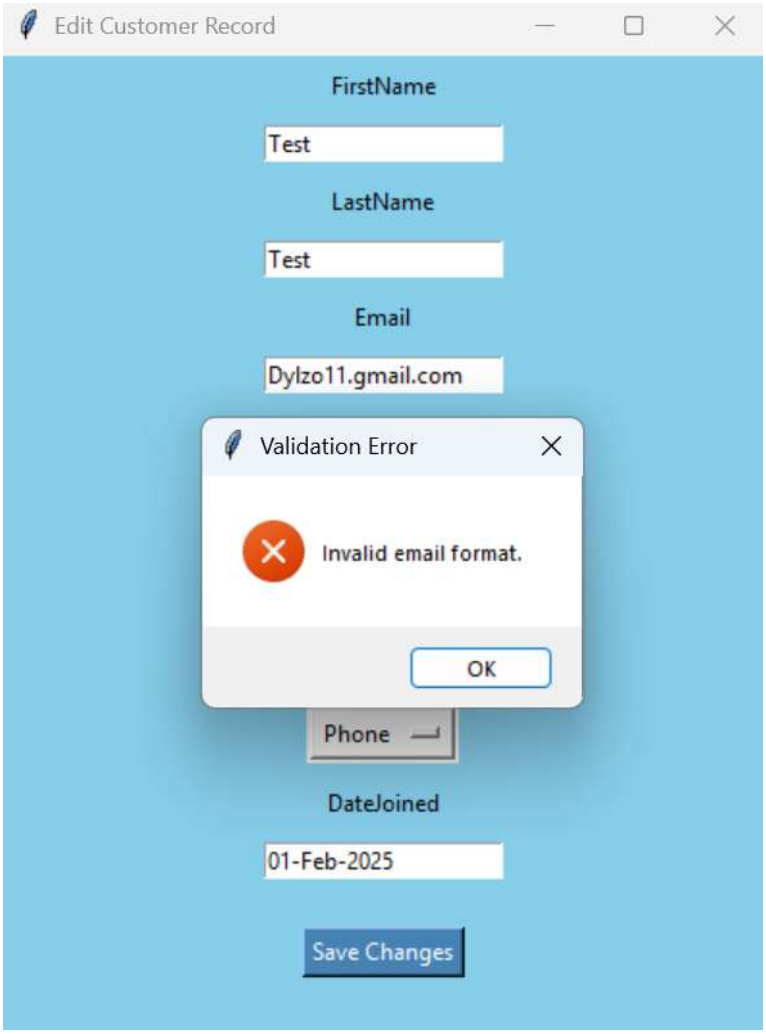


CustomerID	FirstName	LastName	Email	PhoneNumber	Address	PreferredContactMet
3	Test	Test	Ncas@gmail.com###	7512287003	Test	Phone

```
email_pattern = r"^[\w\.-]+@[\w\.-]+\.\w "
```

This was the code ultimately responsible for the validation failure, it uses a regex pattern which I was unfamiliar with and had little resources to fully learn the symbols and their actions.

Solution



As seen, I had successfully gotten my system to recognise the invalid email endings after watching a YouTube video tutorial on email validation

```
email_pattern = r"^[\w\.-]+@[\w\.-]+\.\w+$"
```

```
57      # Email format check for Customer
58      if table_name == "Customer":
59          email = data_dict.get("Email", "").strip()
60          if email:  # Check only if email is provided and not empty
61              if not re.match(email_pattern, email):
62                  return False, "Invalid email format."
```

The issue was that I wasn't using the '$' character in my email validation pattern. What this character does is ensures that no characters can be added on to the end of the validation string. (i.e after @gmail.com). If the user does, it will be flagged and the appropriate error box will appear.